# PERSONAL FINANCE AND EXPENSE ANALYTICS WEB APPLICATION

**A PROJECT REPORT**

***Submitted by***

**ASHUTOSH KUMAR : 24BCS10987**
**ASHUTOSH VERMA : 24BCS10919**

***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

**IN**

COMPUTER SCIENCE ENGINEERING




**Chandigarh University**

OCTOBER 2026

# BONAFIDE CERTIFICATE

Certified that this project report **"PERSONAL FINANCE AND EXPENSE ANALYTICS WEB APPLICATION"** is the bonafide work of "**ASHUTOSH KUMAR and ASHUTOSH VERMA"** who carried out the project work under my supervision.

**SIGNATURE** **SIGNATURE**

**HEAD OF THE DEPARTMENT** **SUPERVISOR**

DEPARTMENT OF COMPUTER
SCIENCE & ENGINEERING
CHANDIGARH UNIVERSITY

Submitted for the project viva-voce examination held on 29 October 2026.

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

# TABLE OF CONTENTS

List of Figures ..... 5

List of Tables ..... 6

List of Standards ..... 7

**CHAPTER 1. INTRODUCTION ..... 12**

1.1. Identification of Client/ Need/ Relevant Contemporary issue ..... 12

1.2. Identification of Problem ..... 12

1.3. Identification of Tasks ..... 13

1.4. Timeline ..... 13

1.5. Organization of the Report ..... 14

**CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY ..... 15**

2.1. Timeline of the reported problem ..... 15

2.2. Existing solutions ..... 15

2.3. Bibliometric analysis ..... 16

2.4. Review Summary ..... 17

2.5. Problem Definition ..... 17

2.6. Goals/Objectives ..... 18

**CHAPTER 3. DESIGN FLOW/PROCESS ..... 19**

3.1. Evaluation & Selection of Specifications/Features ..... 19

3.2. Design Constraints ..... 20

3.3. Analysis of Features and finalization subject to constraints ..... 20

3.4. Design Flow ..... 21

3.5. Design selection ..... 23

3.6. Implementation plan/methodology ..... 24

3.7. Database Schema…….27

3.8. API Endpoints..... 29

**CHAPTER 4. RESULTS ANALYSIS AND VALIDATION ..... 30**

4.1. Implementation of solution ..... 30

**CHAPTER 5. CONCLUSION AND FUTURE WORK ..... 34**

5.1. Conclusion ..... 34

5.2. Future work ..... 35

**REFERENCES ..... 37**

**APPENDIX ..... 39**

1. Plagiarism Report ..... 39

2. Design Checklist ..... 40

**USER MANUAL ..... 41**

# List of Figures

**Figure 1.1 Use Flow ……..... 9**

**Figure 3.1 Development Cycle……..... 27**

# List of Tables

**Table 1.1** Gantt Chart — Project Timeline.....14

**Table 3.1** API Endpoints — Authentication and Transaction Routes.....29

**Table 7.1** Design Checklist.....40

# List of Standards

| Standard | Publishing Agency | About the standard | Page no |
|---|---|---|---|
| IEEE 29148 | IEEE | Defines best practices for Software Requirements Specification (SRS), including functional and non-functional requirements used in the design of the Personal Finance and Expense Analytics Web Application. | Chapter 3 |
| IEEE 829 | IEEE | Specifies standardized software testing documentation practices followed during testing and validation of the web application. | Chapter 4 |

# ABSTRACT

Personal finance management has become increasingly important due to rising expenses and the growing complexity of digital transactions. Many individuals struggle to track their income and expenses effectively and lack proper insights into their financial behavior. To address this issue, this project presents the design and development of a full-stack Personal Finance and Expense Analytics Web Application.

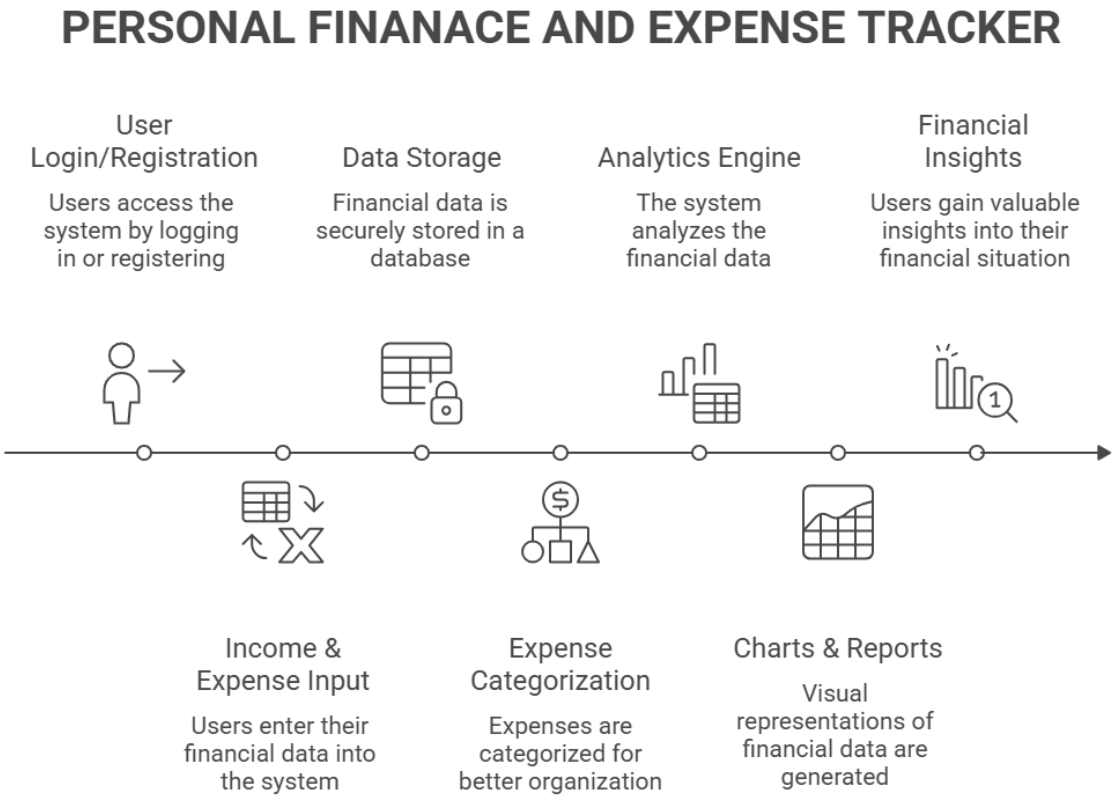
The proposed system is developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) and provides a secure, scalable, and user-friendly platform for managing personal finances. The application includes secure multi-user authentication using JSON Web Tokens (JWT) and bcrypt, ensuring data privacy and isolation for each user.

Users can record, edit, and manage income and expense transactions with custom categories. The system provides interactive analytics such as daily trend analysis, category-wise expense distribution, and monthly summaries using dynamic charts and visualizations. Additionally, a CSV report generation feature allows users to download structured financial reports for monthly and yearly analysis.

The application follows a modular client-server architecture with a React-based Single Page Application (SPA) frontend, a RESTful API backend, and a MongoDB database. The results demonstrate that the system is efficient, secure, and capable of providing meaningful financial insights.

This project promotes better financial awareness and decision-making by offering a modern, analytical, and easy-to-use solution for personal finance management.

# GRAPHICAL ABSTRACT



# ABBREVIATIONS

| ABBREVIATIONS | FULL FORM |
|---|---|
| UI | User Interface |
| UX | User Experience |
| SRS | Software Requirements Specification |
| DB | Database |
| API | Application Programming Interface |
| HTTPS | Hypertext Transfer Protocol Secure |
| CRUD | Create, Read, Update, Delete |
| IEEE | Institute of Electrical and Electronics Engineers |
| MERN | MongoDB, Express.js, React.js, Node.js |
| JWT | JSON Web Token |
| SPA | Single Page Application |
| CSV | Comma- Separated Values |
| REST | Representational State Transfer |
| NPM | Node Package Manager |
| ODM | Object Data Modelling |

# SYMBOLS

| Symbol | Description |
|---|---|
| {} | Curly braces — used in JSON data format for objects |
| [] | Square brackets — used for arrays and IEEE citation references |
| / | Forward slash — used as path separator in API endpoint URLs (e.g., /api/transactions) |
| :id | Colon-prefixed parameter — represents a dynamic route parameter in REST API endpoints |

# CHAPTER 1.
# INTRODUCTION

## 1.1. Identification of Client /Need / Relevant Contemporary issue

In recent years, managing personal finances has become increasingly important due to rising living costs, widespread digital payments, and complex spending patterns. Students and working individuals often struggle to track their income and expenses effectively, which leads to poor saving habits and lack of financial awareness.

Traditional methods such as notebooks and spreadsheets are still commonly used, but they are time-consuming, error-prone, and do not provide meaningful insights. These methods also lack dynamic visualization, making it difficult for users to understand their financial behavior.

There is a strong need for a modern digital solution that not only records financial data but also analyzes and presents it in an easy-to-understand manner. Additionally, in a multi-user environment, it is important to ensure that each user's data remains secure and isolated. The proposed system addresses this need by providing a secure platform with per-user data isolation and interactive visual insights.

## 1.2. Identification of Problem

Despite the availability of digital tools, many individuals still lack access to a simple and efficient system for managing personal finances. Existing solutions either focus only on basic expense recording or are too complex for regular users.

Most traditional tools do not provide real-time analysis, proper categorization, or visual representation of financial data. As a result, users find it difficult to understand their spending patterns and make informed decisions.

Additionally, existing lightweight applications often lack secure per-user data storage, making them unsuitable for multi-user environments. They also do not support features such as inline transaction editing and structured exportable reports.

Therefore, there is a need for a secure, user-friendly, and analytical web-based system that allows users to manage financial data efficiently while providing meaningful insights and data privacy.

## 1.3. Identification of Tasks

To achieve the objectives of the project, the following tasks were identified and carried out:

- Study and analyze existing personal finance management systems
- Identify functional and non-functional requirements of the system
- Design and implement a secure multi-user authentication system using JWT
- Develop income and expense transaction management with full CRUD operations including editing
- Implement custom category creation for personalized financial tracking
- Build real-time monthly analytics with daily trend visualization
- Create a CSV report generation feature for monthly and yearly data export
- Implement and integrate MongoDB Atlas database using Mongoose ODM
- Test the application for accuracy, usability, and responsiveness across devices

These tasks provided a structured approach for the successful development of the application.

## 1.4. Timeline

The project is carried out in a structured manner following a predefined timeline. The major phases include requirement analysis, system design, development, testing, and documentation. A Gantt chart is used to represent the project schedule and track progress throughout the development lifecycle.

| Activity / Week | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8 | Week 9 |
|---|---|---|---|---|---|---|---|---|---|
| Requirement Analysis | | | | | | | | | |
| Literature Review | | | | | | | | | |
| System Design | | | | | | | | | |
| Frontend Development | | | | | | | | | |
| Backend Development | | | | | | | | | |
| Database Design | | | | | | | | | |
| Authentication & Security | | | | | | | | | |
| API Development & Testing | | | | | | | | | |
| Testing & Validation | | | | | | | | | |
| Documentation& Report | | | | | | | | | |

## 1.5. Organization of the Report

This report is organized into five chapters. Chapter 1 introduces the problem, need, objectives, and scope of the project. Chapter 2 presents the literature review and background study of existing solutions. Chapter 3 describes the system design, constraints, and implementation methodology. Chapter 4 discusses the results, analysis, and validation of the developed system. Chapter 5 concludes the report and highlights future enhancements and scope for improvement.

# CHAPTER 2.
# LITERATURE REVIEW/BACKGROUND STUDY

## 2.1. Timeline of the reported problem

Personal finance management has been a concern for individuals for several decades. Earlier, people relied on manual record-keeping methods such as notebooks and spreadsheets to track income and expenses. With the growth of digital banking and online transactions in the early 2000s, the volume of personal financial data increased significantly. However, many individuals still lack proper tools to analyze this data effectively. In recent years, the rise of web and mobile applications has highlighted the need for intelligent financial management systems that can provide analytical insights rather than simple data storage.

## 2.2. Existing solutions

Several solutions have been proposed in the past to address personal finance management. Traditional approaches include manual bookkeeping using notebooks and spreadsheet-based tools such as Microsoft Excel or Google Sheets. While these methods provide flexibility, they are time-consuming, prone to human error, and lack automated analysis and visualization features.

With the advancement of technology, various digital expense tracking applications and budgeting tools have been developed. These applications allow users to record daily expenses, categorize spending, and set budget limits. Some systems also provide basic graphical representations such as pie charts and bar graphs to visualize expenses. However, many of these applications have complex user interfaces, limited customization options, and dependency on third-party services. In addition, concerns related to data privacy, subscription costs, and lack of transparency in data handling restrict their effectiveness.

Overall, existing solutions primarily focus on expense recording rather than detailed

financial analytics and user-controlled insights. These limitations highlight the need for a simple, secure, and analytical web-based personal finance management system.

While modern applications such as Mint and Walnut provide features like automatic bank transaction tracking, they often depend on third-party integrations and raise concerns related to data privacy and security. These applications may also include unnecessary complexity for users who only need basic financial tracking and analysis.

In contrast, the proposed FinTrack system focuses on providing a self-contained and secure solution. It does not rely on external financial services and ensures complete user data privacy through authentication and controlled access. The system is designed using open-source technologies, making it lightweight, transparent, and user-friendly.

## 2.3. Bibliometric analysis

A bibliometric analysis of existing personal finance management systems and related research reveals common trends in features, effectiveness, and limitations as identified in [1], [2], [3]. Most of the reported solutions focus on basic functionalities such as expense recording, income tracking, and budget monitoring. These systems often include categorization of expenses and provide simple visualizations like pie charts and bar graphs to represent spending patterns.

In terms of effectiveness, existing solutions help users develop basic awareness of their spending habits and encourage budgeting discipline. The use of visual analytics improves understanding of financial data and supports short-term decision-making. Some systems also offer notifications and alerts to prevent overspending, which enhances user engagement.

However, several drawbacks are identified through the analysis. Many applications lack advanced analytics and fail to provide meaningful insights beyond basic summaries. Complex user interfaces reduce usability, especially for non-technical users. Additionally, data privacy concerns, limited customization, dependency on third-party platforms, and subscription-based access restrict user control over personal financial data. These limitations reduce the overall effectiveness of existing solutions and indicate the need for a more transparent, user-centric, and analytical web-based finance management system.

## 2.4. Review Summary

The literature review and bibliometric analysis reveal several important findings. First, most existing personal finance management solutions concentrate mainly on basic expense recording and budget tracking, with limited emphasis on detailed financial analytics. Second, although visual representations such as charts are commonly used, they often provide only high-level summaries and fail to offer actionable insights into spending behavior. Third, many systems suffer from complex user interfaces, limited customization options, and concerns related to data privacy and security. Additionally, dependency on third-party services and subscription-based models reduces user control over personal financial data.

These findings indicate a clear gap between user needs and the capabilities of existing solutions. Based on this analysis, the proposed Personal Finance and Expense Analytics Web Application is designed to address these limitations by offering a simple, secure, and user-centric platform. The system focuses on expense categorization, analytical reporting, and intuitive visualization to improve financial awareness and decision-making. Thus, the findings from the literature review directly justify the development and relevance of the proposed project.

## 2.5. Problem Definition

The problem addressed in this project is the lack of an effective, secure, and user-friendly system for managing personal financial data. Many existing tools focus only on recording transactions and fail to provide meaningful insights or advanced functionality.

Additionally, most lightweight solutions lack server-side security and do not support proper data isolation between users. They also do not provide structured exportable reports, which are useful for maintaining offline financial records.

The objective of this project is to develop a web-based Personal Finance and Expense Analytics Application using the MERN stack. The system allows users to securely record income and expenses, categorize transactions, and analyze their financial data through visual insights.

The application includes features such as user authentication, inline transaction editing, custom category creation, and downloadable CSV reports. It focuses on providing a secure, efficient, and easy-to-use solution for personal finance management.

## 2.6. Goals/Objectives

The objectives of this project are defined as measurable goals that guide the development and evaluation of the system.

- To study and understand the fundamentals of personal finance management and expense analytics
- To identify functional and non-functional requirements for a web-based finance application
- To design and implement JWT-based user authentication for secure access and data privacy
- To develop a RESTful API for performing CRUD operations on financial transactions
- To implement expense categorization, including user-defined custom categories
- To build monthly analytics with real-time daily trend visualization using React and Chart.js
- To develop a CSV report generation system for exporting financial data
- To validate the system through testing for accuracy, security, and usability
- To document the complete development process in a structured project report

These objectives ensure that the system is practical, secure, and aligned with real-world requirements.

# CHAPTER 3.
# DESIGN FLOW/PROCESS

## 3.1. Evaluation & Selection of Specifications/Features

Based on the literature review and analysis of existing systems, a set of modern and practical features were selected to design an efficient personal finance management application. The focus was on security, usability, analytics, and real-world applicability.

For this project, the selected features are:

- Secure multi-user registration and login system using JWT and bcrypt
- Income and expense transaction management with full CRUD operations
- Inline transaction editing without requiring deletion and re-entry
- Custom category creation for personalized financial tracking
- Interactive dashboard displaying total income, expenses, net balance, and savings rate
- Monthly analytics with daily trend visualization using line charts
- Category-wise expense distribution using doughnut and bar charts
- Advanced filtering of transactions based on type and category
- CSV report generation for monthly and yearly financial data
- Dark mode and light mode theme support for better user experience
- Responsive design for both desktop and mobile devices

Feature not included in current scope:

- Real time bank account integration
- AI-based financial prediction models
- PDF report generation

These features were excluded due to time constraints and project scope limitations but can be implemented in future versions.

The selected features ensure that the system is not only functional but also secure, scalable, and aligned with modern web application standards.

## 3.2. Design Constraints

The design and development of the Personal Finance and Expense Analytics Web Application are influenced by several practical constraints. These constraints were considered during system planning to ensure feasibility, security, and performance.

- **Technical Constraints:** The application is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system does not include integration with external banking APIs or third-party financial services to maintain simplicity and data privacy.
- **Time Constraints:** The project was completed within a limited academic timeline. Due to this, advanced features such as predictive analytics and machine learning-based recommendations were not included.
- **Security Constraints:** The system uses JWT-based authentication with bcrypt password hashing. Tokens have a limited validity period and must be securely handled on the client side. All protected routes require valid authentication, ensuring secure access control.
- **Resource Constraints:** The application uses MongoDB Atlas free-tier services, which impose certain limitations on storage capacity and performance. Additionally, development was carried out using standard tools and limited infrastructure.
- **Scope Constraints:** The system focuses on transaction tracking, analytics, and CSV report generation. Features such as PDF report export, real-time bank synchronization, and automated financial insights are outside the current scope.

These constraints guided the design decisions and ensured that the system remains practical and achievable.

## 3.3. Analysis of Features and finalization subject to constraints

After evaluating the initially proposed features and considering the design constraints, the system features were refined and finalized to ensure both feasibility and effectiveness.

Unlike initial assumptions, several advanced features such as authentication and multi-user support were successfully implemented in the final system.

Implemented Features:

- Secure user authentication using JWT and bcrypt
- Multi-user support with strict data isolation using unique user ObjectId
- Full CRUD operations for managing income and expense transactions
- Inline editing of transactions, including type switching between income and expense
- Custom category creation for personalized expense tracking
- Monthly analytics with real-time calculations and visualizations
- CSV report generation for exporting financial data

Modified Features:

- Instead of implementing complex predictive analytics, the system focuses on real-time calculated insights such as totals, category breakdowns, and daily trends.
- The user interface is designed as a Single Page Application (SPA) using React for smooth navigation and better user experience.

Excluded Features (Future Scope):

- Real-time bank account integration
- AI-based expense prediction and recommendations
- PDF report generation

The final system successfully balances functionality, performance, and usability while staying within the defined constraints. It delivers a secure, scalable, and user-friendly solution for personal finance management.

## 3.4. Design Flow

The design flow of the system represents the actual working architecture of the application based on a layered MERN stack approach. It ensures secure communication, proper data handling, and efficient analytics generation.

The system follows a structured flow as described below:

**1. User Access**

The user opens the web application through a browser interface.

**2. Authentication Layer**

- New users register by providing full name, username, and password
- Existing users log in using their credentials
- Passwords are securely hashed using bcrypt before storage
- On successful login, a JSON Web Token (JWT) is generated and stored on the client side

**3. Authorization & Middleware**

- All protected API requests include the token in the Authorization header
- The backend middleware verifies the token
- If valid, the user ID is extracted and attached to the request
- Unauthorized requests are blocked with appropriate error responses

**4. Presentation Layer (Frontend – React SPA)**

- The frontend is built as a Single Page Application using React
- Users interact with components such as Dashboard, Add Entry, Transactions, Analytics, and Reports
- Input validation is handled before sending data to the backend

**5. Application Layer (Backend – Express.js)**

- The backend exposes RESTful API endpoints
- Business logic processes incoming requests
- Transactions are categorized as income or expense
- Edit operations intelligently update or shift records between collections

**6. Data Layer (MongoDB Database)**

- Data is stored in MongoDB collections using Mongoose ODM
- Each transaction is linked to a specific user using ObjectId
- Ensures strict data isolation between users

**7. Analytics Layer**

- Financial data is processed to calculate totals, savings, and trends
- Monthly filtering is applied to generate relevant insights
- Category-wise and daily calculations are performed dynamically

**8. Visualization Layer**

- Processed data is converted into charts using Chart.js
- Line charts represent daily trends
- Doughnut and bar charts represent category distributions

**9. Report Generation Layer**

- The system generates structured CSV reports
- Users can download monthly or yearly financial summaries
- Data is formatted for external usage and analysis

**10. Output Display**

- Final results including charts, summaries, and reports are displayed on the user dashboard

This layered design ensures separation of concerns, improves maintainability, and supports scalability for future enhancements.

## 3.5. Design selection

After evaluating different design approaches, a layered architecture based on the MERN stack was selected for implementing the system. This approach provides a clear separation between frontend, backend, and database components, making the system more structured and scalable.

The selection of MERN stack is justified as follows:

- **React.js (Frontend)**: React enables the development of a dynamic Single Page Application (SPA) where components update without reloading the page. This improves user experience and allows smooth navigation between different modules such as Dashboard, Analytics, and Reports.
- **Node.js (Runtime Environment)**: Node.js provides an efficient and asynchronous environment for handling multiple requests, making it suitable for building scalable backend services.
- **Express.js (Backend Framework)**: Express.js simplifies API development by providing a lightweight and flexible routing system. It is used to handle all RESTful API requests and manage application logic.

- **MongoDB (Database):** MongoDB is a NoSQL database that stores data in a flexible document format. It is well-suited for handling user-specific financial records and supports easy scalability.
- **Mongoose (ODM):** Mongoose provides a structured way to define schemas and interact with MongoDB, ensuring data consistency and validation.

This design approach was selected because:

- It provides clear separation of concerns
- It supports secure multi-user architecture
- It enables efficient data handling and analytics
- It allows easy integration of future features
- It follows modern web development standards

Thus, the MERN-based layered architecture ensures that the system is efficient, maintainable, and scalable for real-world applications.

## 3.6. Implementation plan/methodology

The implementation of the Personal Finance and Expense Analytics Web Application follows a structured and modular approach based on the MERN stack architecture. Each component of the system is developed independently and later integrated to form a complete full-stack application.

**1. Frontend Development (React.js with Vite)**

The frontend is developed as a Single Page Application (SPA) using React.js and Vite. It provides a dynamic and responsive user interface.

- Component-based architecture is used for modular design
- Pages include Dashboard, Add Entry, Transactions, Analytics, and Reports
- Axios is used for communication with backend APIs
- Form validation is implemented to ensure correct data entry
- Chart.js is integrated for interactive data visualization
- CSS is used for styling with support for dark and light themes

**2. Backend Development (Node.js and Express.js)**

The backend is implemented using Node.js with Express.js to create a RESTful API.

- API routes are created for authentication and transaction management
- Business logic handles transaction creation, updating, deletion, and retrieval
- Middleware is used to protect routes and validate requests
- Error handling ensures robustness and reliability

**3. Database Design (MongoDB with Mongoose ODM)**

MongoDB is used as the database for storing application data.

- Data is organized into collections such as users, incomes, and expenses
- Each transaction is linked to a user using ObjectId
- Mongoose is used to define schemas and enforce data validation
- The database supports efficient storage and retrieval of records

**4. Authentication System (JWT and Bcrypt)**

A secure authentication mechanism is implemented.

- Passwords are hashed using bcrypt before storage
- Login verifies credentials using hashed comparison
- JWT tokens are generated upon successful authentication
- Tokens are used to access protected routes
- Middleware ensures that only authorized users can access their data

**5. API Integration**

The frontend and backend communicate through REST APIs.

- Axios is configured to automatically attach JWT tokens to requests
- API endpoints handle CRUD operations
- Data is exchanged in JSON format
- This approach ensures modular and clean communication

**6. Expense Categorization**

The system supports both predefined and custom categories.

- Users can select or create categories during transaction entry
- Categories help in organizing financial data
- Category-wise calculations are used in analytics

**7. Analytics and Visualization**

The system provides meaningful financial insights.

- Monthly filtering is applied to transactions
- Daily income and expense trends are calculated
- Category-wise summaries are generated
- Charts such as Line, Doughnut, and Bar charts are used

**8. Testing and Validation**

Testing is performed to ensure system reliability.

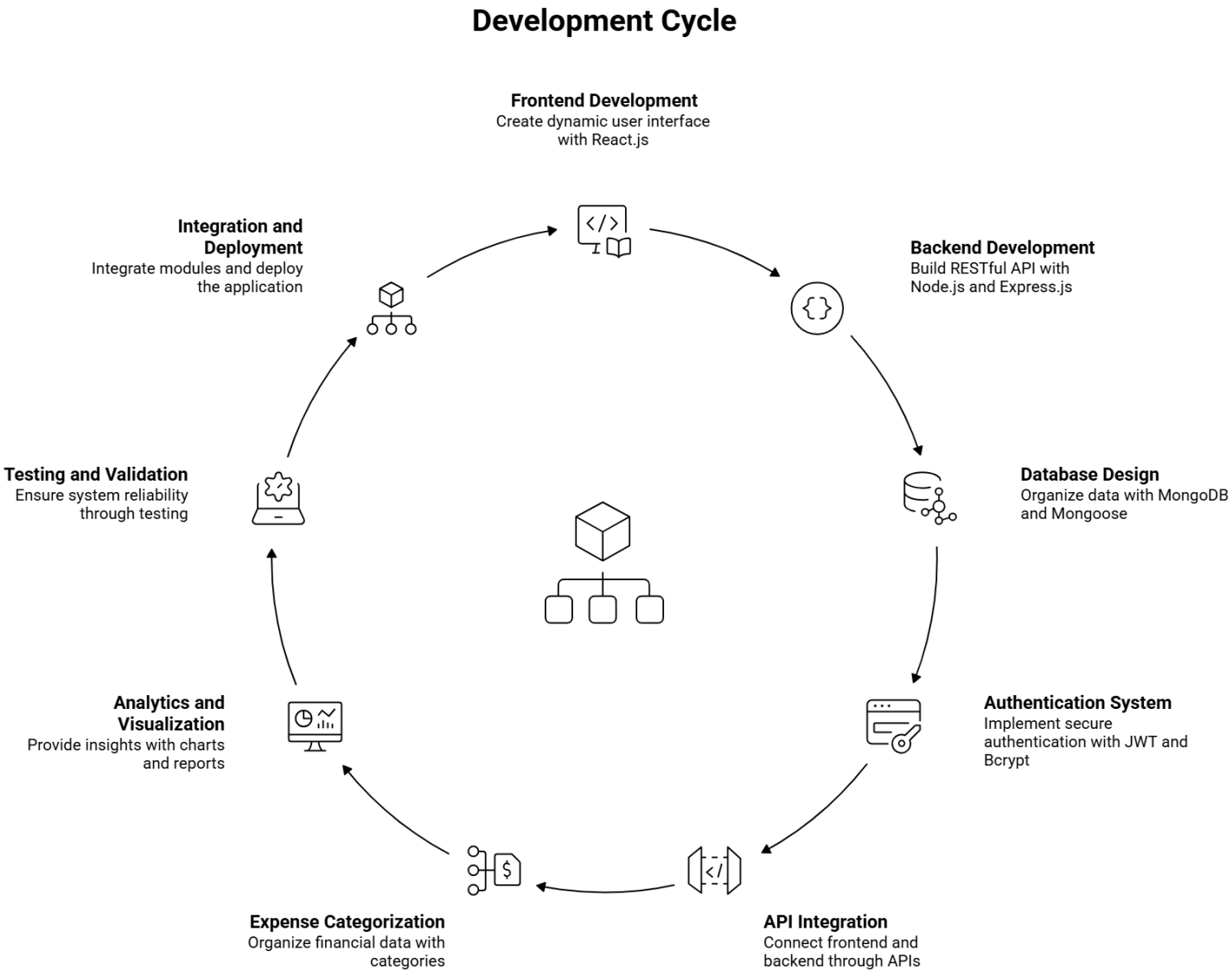
- Input validation prevents incorrect data
- API endpoints are tested for correct responses
- Authentication is tested for secure access
- Multi-user data isolation is verified
- CSV reports are tested for accuracy and format

**9. Integration and Deployment**

All modules are integrated into a complete system.

- Frontend and backend are connected through APIs
- Database connectivity is established
- Application is tested in a local or deployment environment
- Final verification ensures all features work correctly

The implementation follows a clean, modular, and scalable approach, ensuring that the system is efficient, secure, and easy to maintain.




## 3.7. Database Schema

The application uses MongoDB as a NoSQL database, and Mongoose as an Object Data Modeling (ODM) library to define and manage structured schemas. The database is designed with three main collections: User, Income, and Expense.

### 1. User Schema

- fullName — String

- username — String (Unique, Required)
- password — String (Hashed, Required)
- categories — Array of Strings (Custom user-defined categories)

This schema stores user credentials and personalized category data.

**2. Income Schema**

- user — ObjectId (Reference to User)
- amount — Number (Required)
- date — String (YYYY-MM-DD format)
- category — String
- description — String

This schema stores all income-related transactions for a user.

**3. Expense Schema**

- user — ObjectId (Reference to User)
- amount — Number (Required)
- date — String (YYYY-MM-DD format)
- category — String
- description — String

This schema stores all expense-related transactions.

Each transaction document is linked to a specific user using ObjectId. This ensures strict data isolation, meaning users can only access their own financial records. The use of separate collections for income and expense improves clarity and simplifies data processing.

## 3.8. API Endpoints

The backend of the application exposes a set of RESTful API endpoints for handling authentication and transaction management.

Authentication APIs:

| Method | Endpoint | Authentication | Description |
|---|---|---|---|
| POST | /api/auth/register | No | Register a new user |
| POST | /api/auth/login | No | Authenticate user and return JWT |
| POST | /api/auth/categories | Yes | Add custom category |

Transaction APIs (Protected):

| Method | Endpoint | Authentication | Description |
|---|---|---|---|
| GET | /api/transactions | Yes | Retrieve all user transactions |
| POST | /api/transactions | Yes | Add new income/expense |
| PUT | /api/transactions/:id | Yes | Update transaction (supports type change) |
| DELETE | /api/transactions/:id | Yes | Delete transaction |

All protected routes require a valid JWT token. The backend middleware verifies the token and ensures that users can only access their own data.

# CHAPTER 4.
# RESULTS ANALYSIS AND VALIDATION

## 4.1. Implementation of solution

The Personal Finance and Expense Analytics Web Application was successfully implemented using the MERN stack architecture. The system integrates frontend, backend, and database components to provide a secure, scalable, and user-friendly platform for managing personal finances.

The development process involved the use of modern tools and technologies across all phases, including analysis, design, implementation, testing, and communication. The system consists of multiple modules such as authentication, transaction management, analytics, and reporting. Each module was carefully developed and tested to ensure correctness, efficiency, and reliability.

**1. Analysis Phase**

During the analysis phase, system requirements and user needs were identified and structured based on real-world financial tracking problems.

- Requirements were analyzed to define system functionalities
- Data flow and system behavior were planned conceptually
- Feature identification was carried out using structured documentation

This phase provided a clear foundation for system design and implementation.

**2. Design Drawings / Schematics**

The system design was planned using structured architectural approaches.

- A layered architecture (Frontend, Backend, Database) was defined
- Data flow between components was conceptualized
- Flowcharts and diagrams were used to represent system workflow
- API structure and database schema were designed

These design steps helped in organizing the system logic and improving clarity.

### 3. Development Tools and Technologies

Modern development tools were used to build the application efficiently.

- React.js with Vite was used for frontend development
- Node.js and Express.js were used for backend API development
- MongoDB Atlas was used for database management
- Mongoose was used for schema modeling
- Axios was used for API communication
- Chart.js was used for data visualization

These tools enabled the development of a scalable and efficient full-stack system.

### 4. Authentication Results

The authentication system functions correctly using JWT and bcrypt.

- Users can register and log in securely
- Passwords are stored in hashed format
- JWT tokens are generated upon login
- Protected routes validate tokens before access
- Unauthorized access is blocked

This ensures secure multi-user access and data privacy.

### 5. Transaction Management Results

The system supports full CRUD operations for financial transactions.

- Users can add income and expense records
- Transactions can be edited directly without deletion
- Type switching (income ↔ expense) is handled correctly
- Transactions can be deleted efficiently

  All operations are processed accurately and stored in the database.

### 6. Analytics Results

The application provides real-time financial insights.

- Monthly analytics automatically filter data
- Daily trend charts show income and expense patterns
- Category-wise breakdown is displayed using charts
- Summary values such as totals and savings are calculated dynamically

These features help users clearly understand their financial behavior.

**7. Reporting Results**

The system includes a CSV report generation feature.

- Users can generate monthly and yearly reports
- Reports contain structured financial data
- CSV files are correctly formatted
- Download functionality works efficiently

This allows users to maintain external financial records.

**8. User Interface and Theme Results**

The frontend interface is responsive and user-friendly.

- Smooth navigation due to SPA architecture
- Dark mode and light mode toggle works properly
- Layout is optimized for multiple devices

This improves usability and overall experience.

**9. Testing, Validation, and Data Interpretation**

Testing and validation were performed using modern practices.

- API endpoints were tested for correct responses
- Input validation prevents incorrect data entry
- Authentication and authorization were verified
- Multi-user data isolation was confirmed
- Charts and analytics were validated for accuracy
- CSV outputs were tested for correctness

Data interpretation is achieved through visual charts and summaries, helping users analyze financial patterns effectively.

**10. Project Management and Communication**

Project development followed structured management practices.

- Work was divided into phases such as analysis, design, development, and testing
- A Gantt chart was used to track progress
- Tasks were managed systematically
- Team coordination ensured smooth development

Overall Result Summary:

The system successfully implements all intended features, including secure authentication, multi-user data management, real-time analytics, and CSV report generation. The use of modern tools across all phases ensured that the application is efficient, reliable, secure, and easy to maintain. The final system is fully functional and suitable for practical use.

# CHAPTER 5.
# CONCLUSION AND FUTURE WORK

## 5.1. Conclusion

The main objective of this project was to develop a Personal Finance and Expense Analytics Web Application that enables users to manage their financial data efficiently and gain meaningful insights into their spending habits.

The system was designed to provide secure data handling, structured transaction management, and analytical visualization of financial information. The expected outcome was to create a simple and user-friendly application capable of recording income and expenses, categorizing them, and presenting useful insights through charts and summaries.

The actual results achieved not only meet but exceed the initial expectations. The application successfully implements a complete MERN stack architecture with secure multi-user authentication using JWT and bcrypt. It provides full CRUD functionality for transaction management, including inline editing and type switching. The system also includes advanced features such as custom category creation, real-time monthly analytics, and CSV report generation.

The deviations from the initial plan were positive. Features that were originally considered for future work, such as authentication and multi-user support, were successfully implemented within the project timeline. This significantly enhances the overall capability and real-world applicability of the system.

The final application is secure, scalable, and user-friendly. It demonstrates how modern web technologies can be effectively used to build a practical solution for personal finance management. The project also provides a strong foundation for further enhancements and real-world deployment.

## 5.2. Future work

Although the current system provides a comprehensive and functional solution for personal finance management, there are several opportunities for further enhancement to make the application more advanced and suitable for real-world deployment.

1. **PDF Report Generation**

In addition to CSV reports, the system can be extended to generate well-formatted PDF reports for better presentation and sharing.

2. **Budget Planning and Alerts**

Users can be allowed to set monthly spending limits for different categories. The system can provide alerts or visual indicators when spending approaches or exceeds the defined limits.

3. **Recurring Transactions**

Support for recurring income and expenses (such as salary, rent, or subscriptions) can be added to automate data entry and improve convenience.

4. **Receipt Image Upload**

Users can upload images of bills or receipts and attach them to transactions for better record-keeping.

5. **Bank and UPI Integration**

Integration with banking systems or digital payment platforms can allow automatic fetching of transactions, reducing manual input.

6. **Predictive Analytics**

Machine learning techniques can be applied to analyze past data and predict future expenses or savings trends.

7. **Mobile Application Development**

The system can be extended into a mobile application using technologies such as React Native or converted into a Progressive Web App (PWA) for better accessibility.

8. **Enhanced Search and Filtering**

Advanced search options and filters can be implemented to quickly locate specific transactions.

These enhancements will improve the functionality, usability, and scalability of the application, making it more suitable for real-world financial management scenarios.

# REFERENCES


[1] A. Chaudhary, A. Tyagi, A. Sharawat, and G. Chauhan, "Personal Finance Management Web Application," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 11, no. 12, May 2025.

[2] A. Rajitha, A. Sulthana, K. Poojitha, M. Sathwik, and B. V. B. Reddy, "Smart Personal Finance Tracker for Efficient Money Management," *Department of Information Technology, Institute of Aeronautical Engineering*, Hyderabad, India.

[3] M. LakshmiKishore and B. Bhuvaneswari, "AI-Powered Personal Finance Tracker Using Behavioral Pattern Analysis," *International Research Journal of Engineering and Technology (IRJET)*, vol. 12, no. 12, Dec. 2025.

[4] N. Shailendra, S. Gupta, P. C., and R. G. K., "Money Map: The Personal Finance Tracker," *Department of Artificial Intelligence and Data Science, BGSCET (VTU)*, Bengaluru, India.

[5] S. Sharma and R. Verma, "Development of a Web-Based Expense Tracker Using MERN Stack," *International Journal of Computer Science and Engineering (IJCSE)*, vol. 10, no. 4, Apr. 2023.

[6] R. Gupta and P. Mehta, "Secure Authentication in Web Applications Using JSON Web Tokens," *International Journal of Advanced Research in Computer Science (IJARCS)*, vol. 9, no. 2, Mar. 2022.

[7] A. Kumar and S. Singh, "A Study on NoSQL Databases: MongoDB vs. Traditional SQL Databases," *International Journal of Engineering Research and Technology (IJERT)*, vol. 8, no. 6, Jun. 2020.

[8] Meta Open Source, "React — A JavaScript Library for Building User Interfaces," React Official Documentation. [Online]. Available: https://react.dev/ .

[9] OpenJS Foundation, "Node.js Documentation," Node.js Official Documentation. [Online]. Available: https://nodejs.org/en/docs/ .

[10] OpenJS Foundation, "Express.js — Fast, Unopinionated, Minimalist Web Framework for Node.js," Express.js Official Documentation. [Online]. Available: https://expressjs.com/ .

[11] MongoDB, Inc., "MongoDB Documentation — The Developer Data Platform," MongoDB Official Documentation. [Online]. Available: https://www.mongodb.com/docs/ .

[12] Automattic, "Mongoose ODM — Elegant MongoDB Object Modeling for Node.js," Mongoose Official Documentation. [Online]. Available: https://mongoosejs.com/docs/ .

[13] Auth0, "JSON Web Tokens — Introduction and Overview," JWT.io. [Online]. Available: https://jwt.io/introduction .

[14] npm, Inc., "bcryptjs — Package Documentation," npm Registry. [Online]. Available: https://www.npmjs.com/package/bcryptjs .

[15] Evan You, "Vite — Next Generation Frontend Tooling," Vite Official Documentation. [Online]. Available: https://vitejs.dev/ .

[16] Axios Contributors, "Axios — Promise-Based HTTP Client for the Browser and Node.js," Axios Official Documentation. [Online]. Available: https://axios-http.com/docs/intro .

[17] Chart.js Contributors, "Chart.js — Simple Yet Flexible JavaScript Charting," Chart.js Official Documentation. [Online]. Available: https://www.chartjs.org/docs/ .

[18] MDN Web Docs, "HTML, CSS, and JavaScript Documentation," Mozilla Developer Network. [Online]. Available: https://developer.mozilla.org/ .

[19] N. Neeraj, "Personal Finance Tracker," GitHub Repository. [Online]. Available: https://github.com/neeraj542/Personal-Finance-Tracker .

# APPENDIX

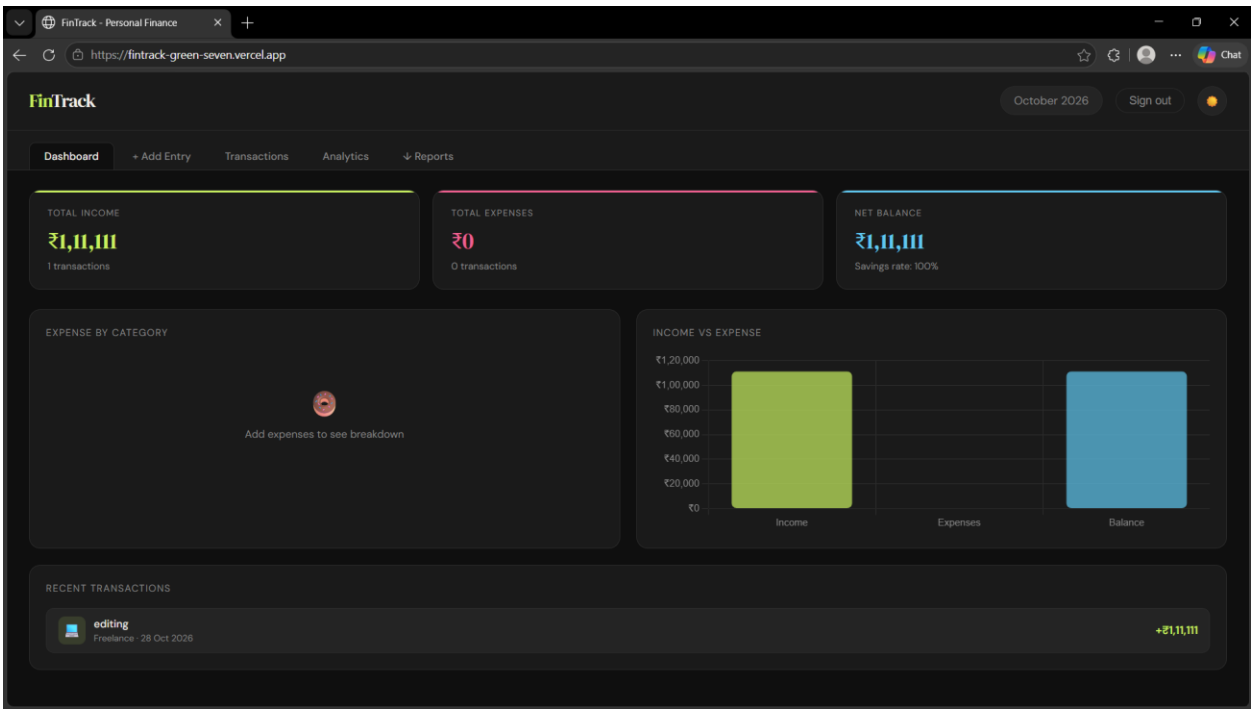
## 1. Design Checklist

| | | | |
|---|---|---|---|
| 1 | Problem Identification | Problem clearly defined | ✓ |
| 2 | Requirement Analysis | Functional & non-functional requirements identified | ✓ |
| 3 | Literature Review | Existing solutions studied | ✓ |
| 4 | Feature Selection | Relevant features selected | ✓ |
| 5 | Design Constraints | Constraints identified and considered | ✓ |
| 6 | System Design | Flowcharts and diagrams prepared | ✓ |
| 7 | Technology Selection | Suitable technologies chosen | ✓ |
| 8 | Implementation | Core features implemented | ✓ |
| 9 | Testing | System tested with test cases | ✓ |
| 10 | Validation | Output verified and validated | ✓ |
| 11 | Documentation | Report prepared as per guidelines | ✓ |

# USER MANUAL

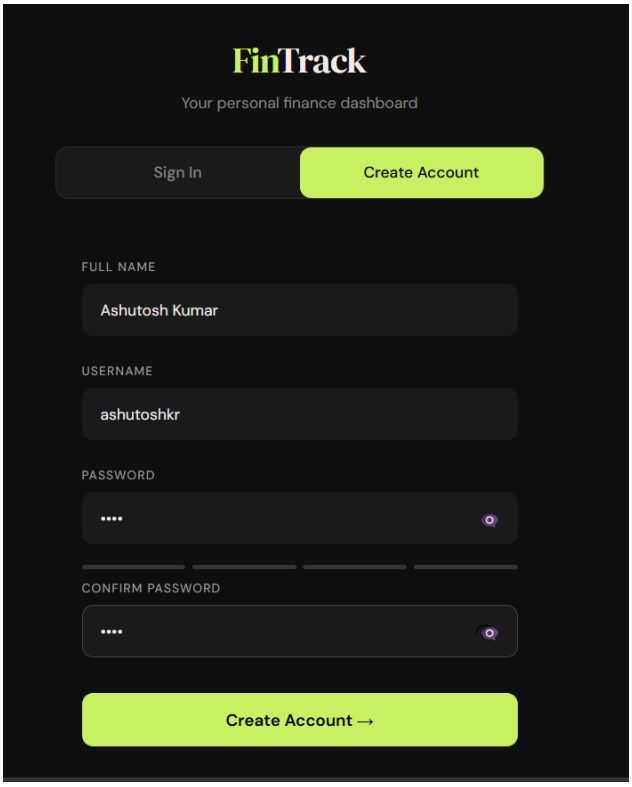
## 1. Accessing the Application:

- Open any modern web browser on your device. Recommended browsers: Google Chrome, Mozilla Firefox or Microsoft Edge (latest version).
- Type or paste the following deployed application URL and press Enter: https://fintrack-green-seven.vercel.app/
- The FinTrack authentication screen will load automatically. No installation required.



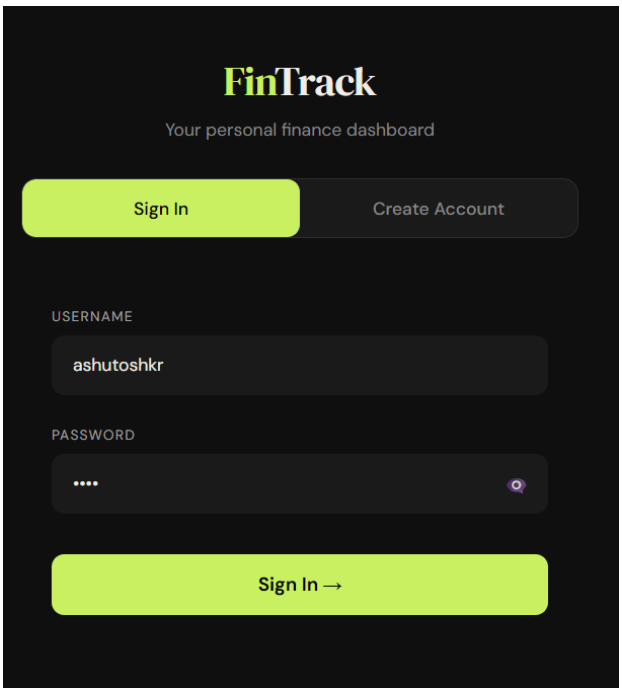

## 2. Creating a new Account:

- On the authentication screen, click the "Create Account" option to switch to the registration form.
- Enter the details: Name, Username, Password
- Click Create Account
- You will be logged in and redirected to the Dashboard.



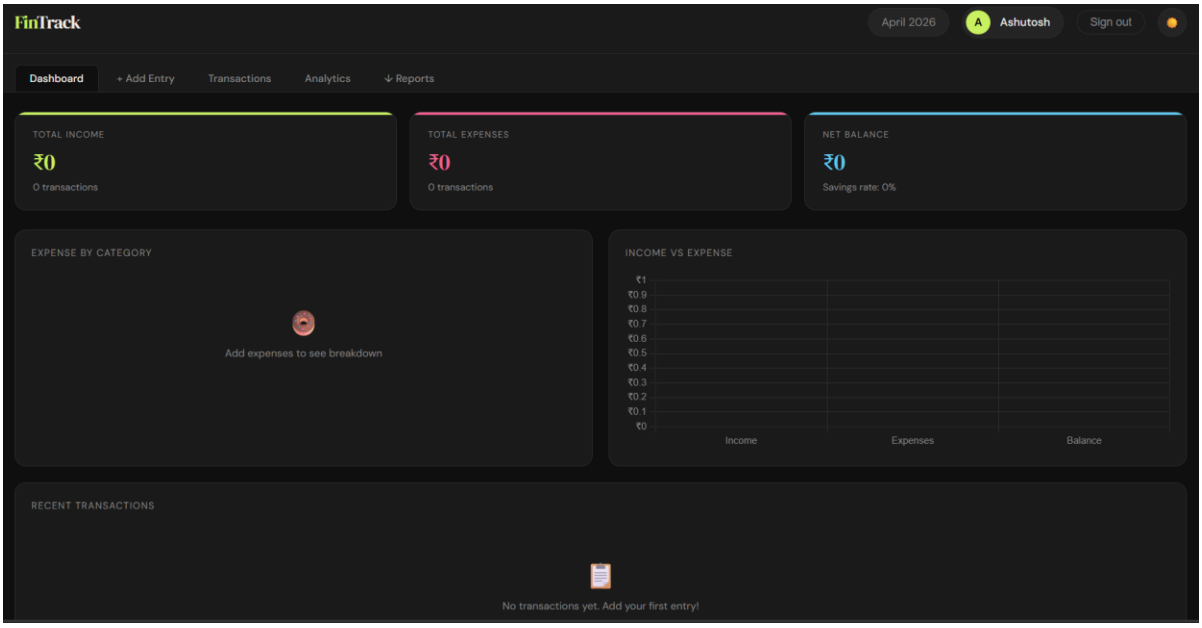

## 3. Login

- Enter username and password
- Click Login



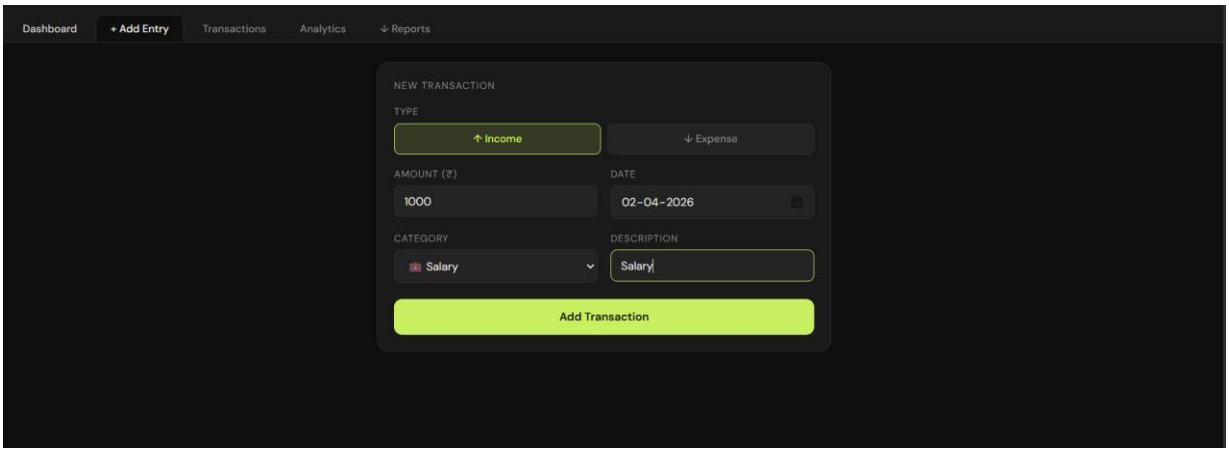
## 4. Dashboard Overview

- After login,, the Dashboard shows Total Income, Total Expenses, Net Balance
- Charts included: Expense by Category, Income vs Expense Bar Chart



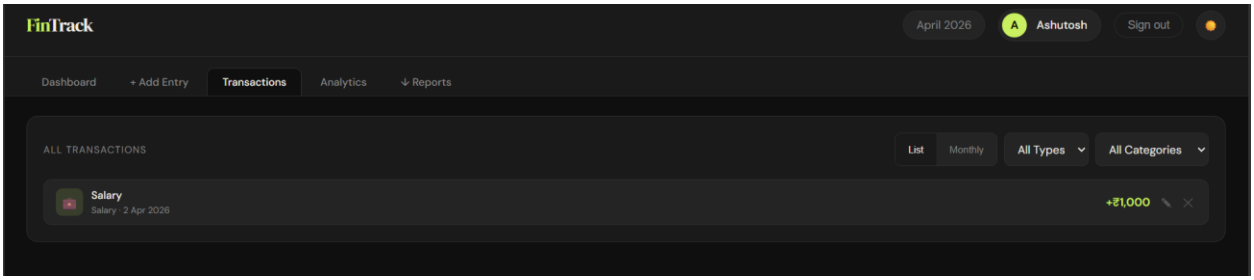

## 5. Add Transaction

- Click "+ Add Entry".
- Fill details: Type (Income/Expense), Amount, Date, Category (or create new), Description
- Click "Add Transaction".
- Data updates instantly in Dashboard and Analytics.



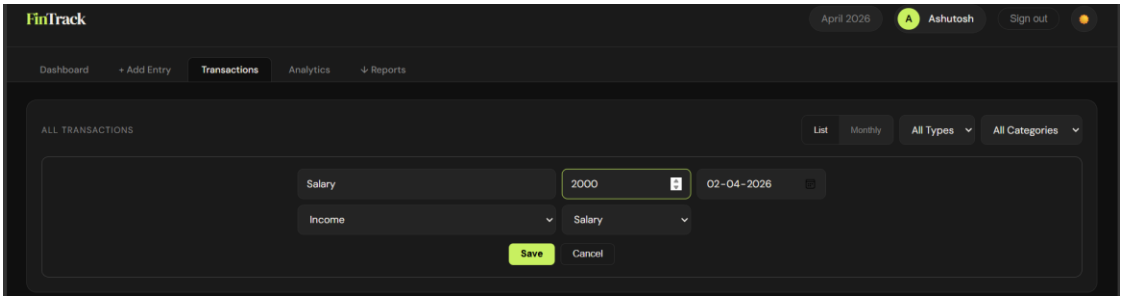
## 6. View Transactions

- Click “Transactions”.
- View all records (latest first).



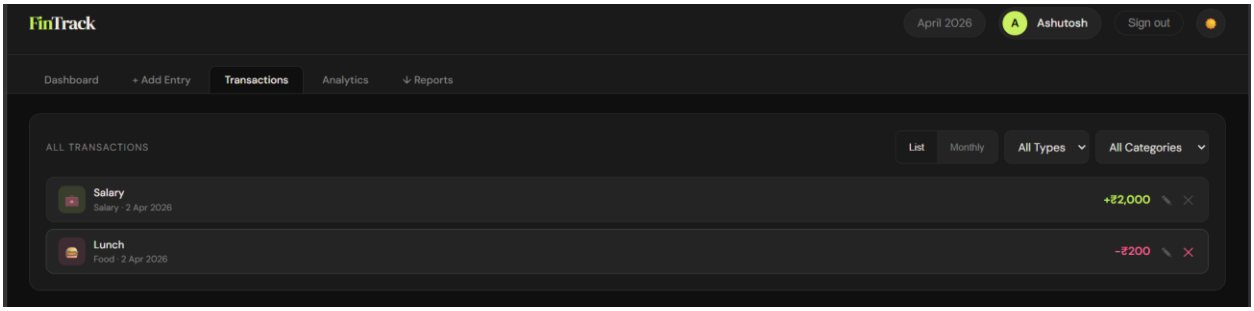

## 7. Edit Transactions

- Click edit ( icon).
- Update details.
- Click “Save” or “Cancel”.



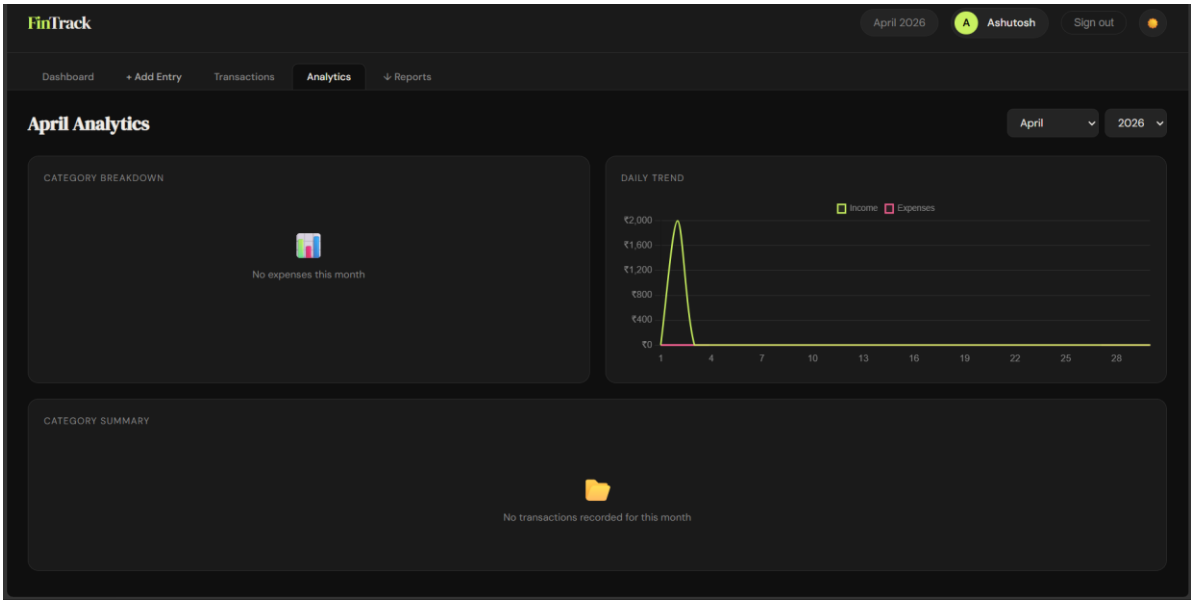

## 8. Delete Transactions

- Click delete ( icon).
- Transaction is removed instantly.
- Dashboard updates automatically.



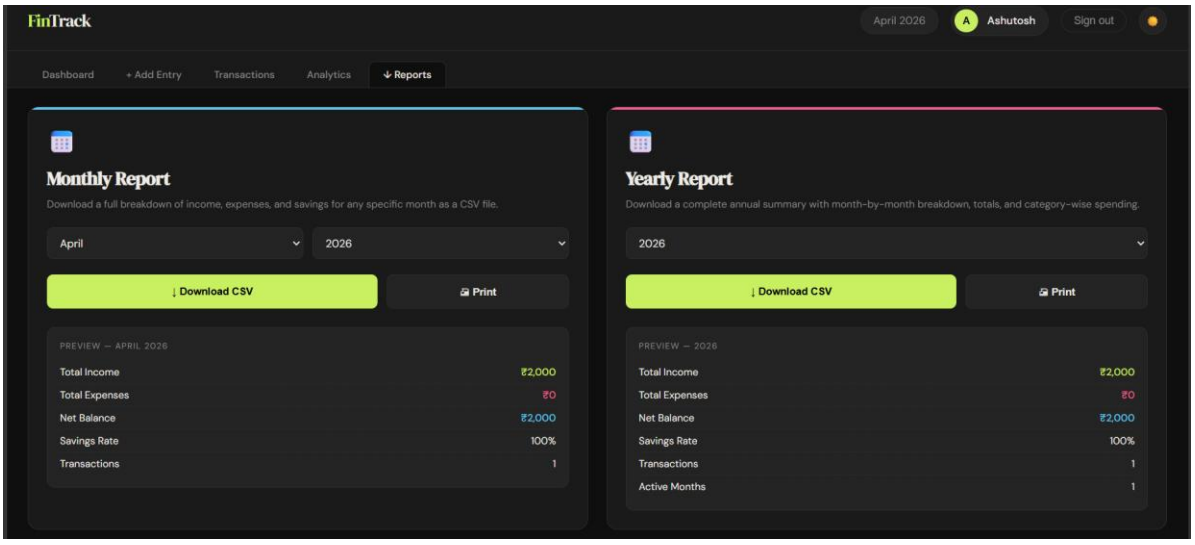
## 9. Analytics

- Click “Analytics”.
- Select Month and Year.



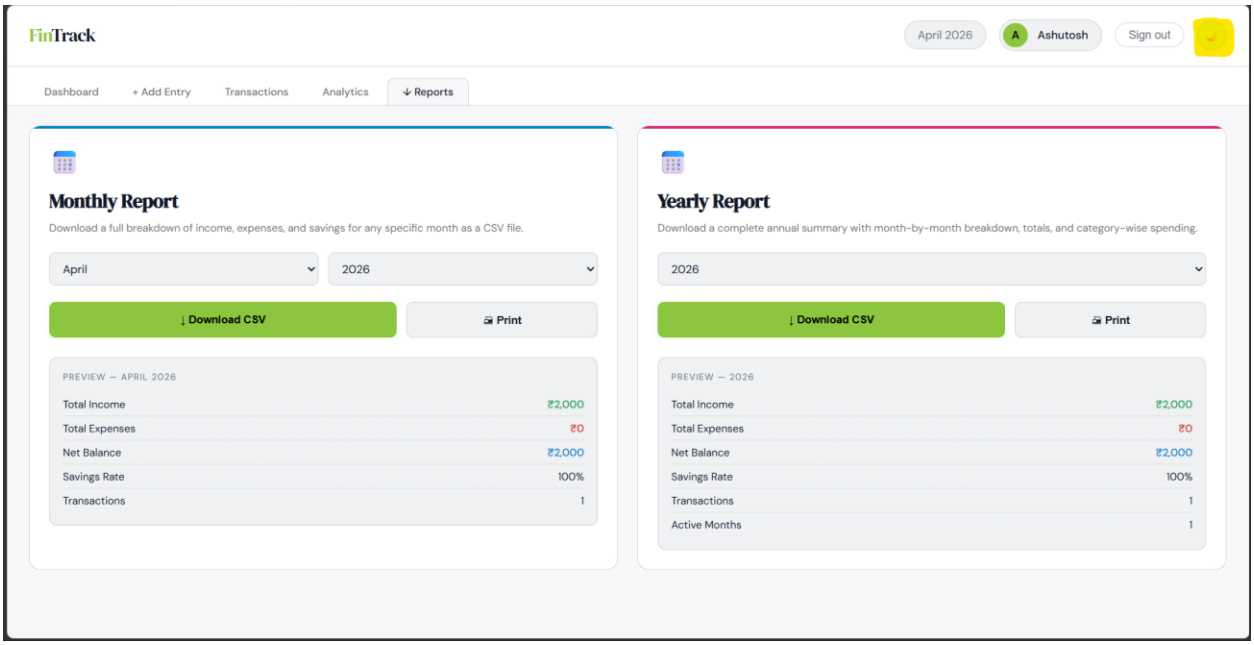

## 10. Download Reports

- Go to “Reports”.
- Monthly Report: Select month & year; Click “Download CSV”
- Yearly Report: Select year; Download CSV
- Files open in Excel or Google Sheets.



## 11. Theme Mode

- Click the theme icon (☀ / 🌙).
- Switch between Dark and Light mode.




## 12. Logout

- Click “Sign Out”.
- You will return to the login page.
- Your data remains secure in database.

